01 PRACTICAL 1

ASSIGNMENT 4 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

from gensim.models import Word2Vec

from tensorflow.keras import models, layers

df = pd.read_csv("IMDB Dataset.csv")

print(df.head())
print(df.shape)

# CLEAN AND TOKENIZE REVIEWS
def clean_text(text):
    text = text.lower()
    text = re.sub("[^a-zA-Z ]", "", text)

    words = text.split()
    words = [word for word in words if word not in ENGLISH_STOP_WORDS ]

    return words

sentences = df['review'].apply(clean_text).tolist()

print(sentences[0])

# ENCODE LABELS
encoder = LabelEncoder()

y = encoder.fit_transform(df['sentiment'])

# SPLITING THE DATASET
X_train, X_test, y_train, y_test = train_test_split(
    sentences, y, random_state = 42, test_size = 0.2
)

print("Training size:", len(X_train))
print("Testing size:", len(X_test))

# TRAINING CBOW WORD2VEC MODEL
w2v = Word2Vec(
    X_train,
    vector_size = 100,
    window = 5,
    min_count = 2,
    sg = 0 ,
    epochs = 10
)

print("Vocabulary size: ", len(w2v.wv))

print(w2v.wv["movie"])

# CONVERT REVIEW INTO VECTOR EMBEDDINGS
def get_vector(words):
    vectors = []
```

```
    for word in words:
        if word in w2v.wv:
            vectors.append(w2v.wv[word])

    if len(vectors) == 0 :
        return np.zeros(100)

    return np.mean(vectors, axis = 0 )


X_train = np.array([get_vector(s) for s in X_train])
X_test = np.array([get_vector(s) for s in X_test])

print(X_train[0])

# DEFINING THE MODEL ARCHITECTURE
model = models.Sequential([
    layers.Dense(64, activation="relu", input_shape=(100,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.summary()

# COMPILING AND TRAINING THE MODEL
model.compile(
    optimizer = "adam",
    loss = "binary_crossentropy",
    metrics=['accuracy']
)

history = model.fit( X_train, y_train, epochs = 11, batch_size = 32 , validation_split
= 0.2)

# EVALUATE MODEL
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# Accuracy graph
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Loss graph
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

#Predict and Display Actual vs Predicted Sentiment
predictions = model.predict(X_test)

y_pred = (predictions > 0.5).astype(int)

print("Predicted:", y_pred[:10].flatten())
```

```
print("Actual:", y_test[:10])
```

02 PRACTICAL 2

ASSIGNMENT 4 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from gensim.models import Word2Vec

df = pd.read_csv("twitter_dataset.csv")

print(df.head())
print(df.shape)

# CLEAN AND TOKENIZE TWEETS
def clean_text(text):
    text = text.lower()

    text = re.sub("[^a-zA-Z ]", "", text)

    words = text.split()

    words = [word for word in words if word not in ENGLISH_STOP_WORDS]

    return words

sentences = df['Text'].dropna().apply(clean_text).tolist()

print(sentences[0])

# TRAIN CBOW Word2Vec MODEL
model = Word2Vec(
    sentences = sentences,
    vector_size = 100,
    window = 5,
    min_count = 2,
    epochs = 10
)

print("Vocabular size: ", len(model.wv))

# EVALUATE THE RESUL
word = "happy"

print("Word Embedding:")
print(model.wv[word])

print("Embedding shape: ", model.wv[word].shape)

# FIND SIMILAR WORDS
similar_words = model.wv.most_similar("happy", topn=5)

print(similar_words)

print("Similarity (money, economy):", model.wv.similarity("money", "economy"))
```

03 PRACTICAL 3

ASSIGNMENT 4 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import re

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

from gensim.models import Word2Vec

# LOADING THE DATASET
with open("document3.txt", "r", encoding="utf-8") as file:
    text = file.read()

print(text)

# CLEANING THE DATASET AND TOKENIZING
text = text.lower()

text = re.sub(r"[^a-z ]", "", text)

words = text.split()

words = [ word for word in words if word not in ENGLISH_STOP_WORDS ]

print(words[1])

# TRAIN THE CBOW MODEL
w2v = Word2Vec(
    [words],
    vector_size = 100,
    window = 5,
    min_count = 1,
    sg = 0,
    epochs = 100
)

print("Vocabulary size: ", len(w2v.wv))

# EVALUATE THE MODEL
word = words[2]

print("Word:", word)
print("Embedding:", w2v.wv[word])
print("Embedding Shape:", w2v.wv[word].shape)

print("Total words:", len(words))
print("Unique words:", len(set(words)))
print("Vocabulary size:", len(w2v.wv))

print("Similar to 'covid':", w2v.wv.most_similar("covid", topn=5))
print("Similar to 'influenza':", w2v.wv.most_similar("influenza", topn=5))
print("Similarity (covid, influenza):", w2v.wv.similarity("covid", "influenza"))
```

04 PRACTICAL 4

ASSIGNMENT 4 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import re

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from gensim.models import Word2Vec

from sklearn.metrics.pairwise import cosine_similarity

df = pd.read_csv("twitter_dataset.csv")

df.head()

# CLEAN AND TOKENIZE TWEETS
def clean_text(text):
    text = text.lower()

    text = re.sub(r"[^a-z ]", "", text )

    words = text.split()

    words = [word for word in words if word not in ENGLISH_STOP_WORDS]

    return words

tweets = df['Text'].dropna().astype(str)

sentences = tweets.apply(clean_text).tolist()

print(sentences[0])

# TRAIN WORD2VEC MODEL
w2v = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=2,
    sg=0,
    epochs=10
)

print("Vocabulary Size:", len(w2v.wv))

# CONVERT TWEETS INTO DOCUMENT VECTOR
def get_vector(words):
    vectors = [w2v.wv[w] for w in words if w in w2v.wv]

    if len(vectors) == 0 :
        return np.zeros(100)

    return np.mean(vectors, axis = 0 )

tweet_vectors = np.array([get_vector(s) for s in sentences])

print("Document Vectors Shape: ", tweet_vectors.shape)

# TAKE USER INPUT
user_input = input("Enter keywords or a tweet: ")

query_words = clean_text(user_input)
```

```
query_vector = get_vector(query_words)

print(user_input)
print(query_vector)

# CALCULATE COSINE SIMILARITY
similarity = cosine_similarity(
    [query_vector],
    tweet_vectors
)[0]

# RECOMMENT TOP 5 SIMILAR TWEETS
top_indices = np.argsort(similarity)[::-1][:5]

for i in top_indices:
    print("\nTweet: ", tweets.iloc[i])
    print("Similarity: ", similarity[i])
```